

# Audio Signal Analyzer & Noise Reduction System

Complete Beginner's Study Guide: 15 Core Concepts Taught From Zero with Code Mappings & Viva Answers

---

## Concept 1: Audio Signal

|                       |                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Simple Explanation:   | Sound is vibration traveling through the air as continuous pressure waves. In computers, an audio signal is represented as numbers indicating how strongly air is being compressed or rarefied at each instant. A pure tone is a smooth repeating wave (sine wave), while real speech or instruments are mixtures of many waves added together. |
| Project Code Mapping: | <code>audio_io.py -&gt; load_audio(filepath) &amp; to_mono(data)</code>                                                                                                                                                                                                                                                                         |
| Why We Use It:        | We must import sound files (WAV) into Python and convert them into 1D numerical arrays of floating-point numbers between [-1.0, 1.0] so our math functions can process them.                                                                                                                                                                    |
| Viva Exam Q&A::       | <b>Q:</b> What is an audio signal physically and digitally? Physically, an audio signal is a continuous acoustic pressure wave. Digitally, it is a sequence of discrete numerical amplitude values representing that pressure over time.                                                                                                        |

## Concept 2: Sampling

|                       |                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Simple Explanation:   | A computer cannot record every infinite millisecond of continuous time. Instead, it takes 'snapshots' (measurements) of sound thousands of times per second—exactly like taking 60 video frames per second to create motion picture. The number of snapshots per second is the Sampling Frequency (fs). Standard CD-quality audio uses $f_s = 44,100$ Hz. |
| Project Code Mapping: | <code>audio_io.py -&gt; load_audio (reads fs) &amp; sample_audio/generate_samples.py</code>                                                                                                                                                                                                                                                               |
| Why We Use It:        | Sampling converts continuous real-world acoustic waves into discrete numbers that a digital computer can store, analyze, and manipulate.                                                                                                                                                                                                                  |
| Viva Exam Q&A::       | <b>Q:</b> What is the Nyquist-Shannon Sampling Theorem? It states that the sampling rate $f_s$ must be strictly greater than twice the highest frequency present in the signal ( $f_s > 2 * f_{max}$ ) to prevent distortion known as aliasing.                                                                                                           |

### Concept 3: Discrete-Time Signal

|                              |                                                                                                                                                                                                                                                                                                                   |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple Explanation:</b>   | A discrete-time signal is written mathematically as $x[n]$ , where $n$ is an integer index ( $n = 0, 1, 2, \dots, N-1$ ). Unlike continuous time $t$ which has infinite decimals, $n$ just counts the samples: sample 0, sample 1, sample 2, etc. The actual physical time of sample $n$ is simply $t = n / fs$ . |
| <b>Project Code Mapping:</b> | <code>signal_processing.py -&gt; compute_time_axis(num_samples, fs)</code>                                                                                                                                                                                                                                        |
| <b>Why We Use It:</b>        | Computers can only perform arithmetic on discrete memory arrays. Our time vector $t[n] = n / fs$ allows us to plot real seconds on graph axes.                                                                                                                                                                    |
| <b>Viva Exam Q&amp;A::</b>   | <b>Q:</b> How is continuous time related to sample index $n$ ? Continuous time is related by $t = n * Ts = n / fs$ , where $Ts$ is the sampling period and $fs$ is the sampling frequency.                                                                                                                        |

### Concept 4: Time Domain

|                              |                                                                                                                                                                                                                                                                                                                                       |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple Explanation:</b>   | The time domain is the view of a signal where the horizontal axis is Time (seconds) and the vertical axis is Amplitude (volume/voltage). Looking at the time domain shows you WHEN things happen (a drum hit, silence, or sudden loud noise), but it cannot tell you which musical notes or individual pitch frequencies are playing. |
| <b>Project Code Mapping:</b> | <code>gui.py -&gt; _render_waveform_axis(...)</code> & <code>signal_processing.py -&gt; compute_signal_stats()</code>                                                                                                                                                                                                                 |
| <b>Why We Use It:</b>        | Visualizing the time-domain waveform lets us verify signal amplitude, spot clipping ( $> 1.0$ ), inspect envelope shapes, and see random noise fluctuations.                                                                                                                                                                          |
| <b>Viva Exam Q&amp;A::</b>   | <b>Q:</b> What are the limitations of analyzing an audio signal purely in the time domain? The time domain only shows amplitude variations over time; it cannot reveal the individual frequency components or musical pitches that compose the signal.                                                                                |

### Concept 5: Frequency Domain

|                              |                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple Explanation:</b>   | The frequency domain decomposes a complex wave into its recipe of pure ingredients. The horizontal axis is Frequency (Hz) and the vertical axis is Magnitude (strength). For example, if you play an A-major chord on a piano, the time domain looks like messy vibrations, but the frequency domain shows three clear vertical spikes corresponding to the notes A, C#, and E. |
| <b>Project Code Mapping:</b> | <code>gui.py -&gt; _render_fft_axis(...)</code> & <code>signal_processing.py -&gt; compute_fft()</code>                                                                                                                                                                                                                                                                         |
| <b>Why We Use It:</b>        | Noise and unwanted sounds usually occupy specific frequency ranges. We must look at the frequency domain to pinpoint where the noise lives so we know where to cut it.                                                                                                                                                                                                          |
| <b>Viva Exam Q&amp;A::</b>   | <b>Q:</b> Why do we transform signals into the frequency domain? Because many signal operations—such as identifying pitch, detecting interference tones, and designing frequency-selective filters—are simple and intuitive in the frequency domain but impossible in the time domain.                                                                                          |

## Concept 6: Discrete Fourier Transform (DFT)

|                              |                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple Explanation:</b>   | The Discrete Fourier Transform is the exact mathematical equation that converts N samples in the time domain $x[n]$ into N frequency points $X[k]$ . It works by multiplying the signal with complex sine and cosine waves of every possible frequency and adding up the results. If a frequency exists in the audio, the sum is large; if not, it cancels to zero. |
| <b>Project Code Mapping:</b> | <code>signal_processing.py -&gt; compute_fft</code> (mathematical foundation: $X[k] = \sum(x[n] * e^{(-j*2*\pi*k*n/N)})$ )                                                                                                                                                                                                                                          |
| <b>Why We Use It:</b>        | DFT is the formal bridge between discrete time and discrete frequency in computer digital signal processing.                                                                                                                                                                                                                                                        |
| <b>Viva Exam Q&amp;A::</b>   | <i>Q: What is the computational complexity of the direct DFT? Calculating direct DFT requires <math>O(N^2)</math> complex multiplication and addition operations, which is too slow for real-time processing of long audio files.</i>                                                                                                                               |

## Concept 7: Fast Fourier Transform (FFT)

|                              |                                                                                                                                                                                                                                                                                                                   |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple Explanation:</b>   | The FFT is NOT a different transform; it is an ingenious fast algorithm that calculates the EXACT same DFT in a fraction of the time (using divide-and-conquer). For 100,000 samples, direct DFT requires 10 billion calculations, while FFT takes only about 1.6 million calculations—thousands of times faster! |
| <b>Project Code Mapping:</b> | <code>signal_processing.py -&gt; compute_fft(signal, fs)</code> using <code>np.fft.rfft</code>                                                                                                                                                                                                                    |
| <b>Why We Use It:</b>        | We use <code>np.fft.rfft</code> because audio is real-valued. It computes only the positive frequencies up to Nyquist ( $fs / 2$ ) with normalized physical magnitude.                                                                                                                                            |
| <b>Viva Exam Q&amp;A::</b>   | <i>Q: Why do we use rfft instead of standard fft? Because real-valued audio signals have symmetric Fourier transforms (Hermitian symmetry). rfft computes only the unique positive half of the spectrum (0 to <math>fs/2</math>), saving 50% CPU time and memory.</i>                                             |

## Concept 8: Noise (AWGN)

|                              |                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple Explanation:</b>   | Noise is any unwanted electrical, acoustic, or digital disturbance that corrupts the signal. In this project we use Additive White Gaussian Noise (AWGN): 'Additive' means it simply adds on top of our sound; 'White' means it spreads uniformly across all frequencies like a flat hissing floor; 'Gaussian' means random noise amplitudes follow a standard bell curve. |
| <b>Project Code Mapping:</b> | <code>signal_processing.py -&gt; add_white_gaussian_noise(signal, target_snr_db, seed)</code>                                                                                                                                                                                                                                                                              |
| <b>Why We Use It:</b>        | We simulate realistic degradation in a controlled, reproducible manner so we can objectively test how effectively our digital filters remove noise.                                                                                                                                                                                                                        |
| <b>Viva Exam Q&amp;A::</b>   | <i>Q: Why is it called 'White' noise? Just as white light contains equal intensities of all visible colors/wavelengths, white noise contains equal power across all frequency bands in the spectrum.</i>                                                                                                                                                                   |

## Concept 9: Low-Pass Filter (LPF)

|                              |                                                                                                                                                                                                                                                                                            |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple Explanation:</b>   | A low-pass filter acts like an acoustic sieve: it lets low frequencies pass through unaffected (bass, human fundamental voice) while blocking and attenuating all high frequencies above a chosen cutoff (high-pitched hiss, whistles, screeching noise).                                  |
| <b>Project Code Mapping:</b> | <code>signal_processing.py -&gt; apply_lowpass_filter(signal, cutoff, fs, order=4)</code>                                                                                                                                                                                                  |
| <b>Why We Use It:</b>        | When high-frequency noise or unwanted high tones (like our 3500 Hz tone) contaminate a low-frequency sound (like our 300 Hz tone), a low-pass filter eliminates the high noise.                                                                                                            |
| <b>Viva Exam Q&amp;A::</b>   | <b>Q:</b> <i>What is the ideal passband and stopband behavior of a low-pass filter? The passband passes all frequencies from 0 to <math>f_c</math> with a gain of 1 (0 dB), while the stopband completely eliminates all frequencies above <math>f_c</math> with infinite attenuation.</i> |

## Concept 10: High-Pass Filter (HPF)

|                              |                                                                                                                                                                                                                               |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple Explanation:</b>   | The opposite of low-pass: a high-pass filter blocks all low frequencies below the cutoff (such as low AC electrical hum at 50/60 Hz, microphone thumps, wind rumble) while allowing high frequencies to pass through cleanly. |
| <b>Project Code Mapping:</b> | <code>signal_processing.py -&gt; apply_highpass_filter(signal, cutoff, fs, order=4)</code>                                                                                                                                    |
| <b>Why We Use It:</b>        | Used to remove low-frequency DC offset and low-frequency rumble without altering the treble or high harmonics of speech.                                                                                                      |
| <b>Viva Exam Q&amp;A::</b>   | <b>Q:</b> <i>Give an everyday application of a high-pass filter. Removing low-frequency wind rumble, breathing pops, or 50/60 Hz electrical power-line hum from microphone vocal recordings.</i>                              |

## Concept 11: Band-Pass Filter (BPF)

|                              |                                                                                                                                                                                                                                                                                                                                              |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple Explanation:</b>   | A band-pass filter allows only a specific window of frequencies between a lower cutoff ( $f_{low}$ ) and an upper cutoff ( $f_{high}$ ) to pass through. It blocks both very low rumble below $f_{low}$ AND very high hiss above $f_{high}$ . A classic example is a telephone channel, which only passes speech between 300 Hz and 3400 Hz. |
| <b>Project Code Mapping:</b> | <code>signal_processing.py -&gt; apply_bandpass_filter(signal, low_cutoff, high_cutoff, fs, order=4)</code>                                                                                                                                                                                                                                  |
| <b>Why We Use It:</b>        | Used when the desired audio signal occupies an intermediate frequency band and is surrounded by both low-end and high-end noise.                                                                                                                                                                                                             |
| <b>Viva Exam Q&amp;A::</b>   | <b>Q:</b> <i>How are the cutoff frequencies of a band-pass filter defined? A band-pass filter has two cutoff frequencies: <math>f_{low}</math> (lower cutoff) and <math>f_{high}</math> (upper cutoff), which define the passband bandwidth <math>B = f_{high} - f_{low}</math>.</i>                                                         |

## Concept 12: Cutoff Frequency (fc)

|                              |                                                                                                                                                                                                                                                                            |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple Explanation:</b>   | The cutoff frequency is the boundary line between the frequencies we keep and the frequencies we discard. On a filter's frequency graph, the cutoff frequency is defined as the point where signal power drops to 50% (which equals an amplitude drop to 70.7%, or -3 dB). |
| <b>Project Code Mapping:</b> | <code>signal_processing.py -&gt; validate_cutoff_frequency(cutoff, fs, filter_type)</code>                                                                                                                                                                                 |
| <b>Why We Use It:</b>        | Our software strictly checks that $0 < f_c < f_s / 2$ (Nyquist limit). A digital filter cannot have a cutoff frequency above half the sampling rate.                                                                                                                       |
| <b>Viva Exam Q&amp;A::</b>   | <i><b>Q:</b> Why is the cutoff frequency called the -3 dB point? Because -3 dB corresponds to <math>20 * \log_{10}(1 / \sqrt{2}) \approx -3.01</math> dB, which is the exact frequency where output signal power drops to half of its input power.</i>                     |

## Concept 13: Butterworth Filter

|                              |                                                                                                                                                                                                                                                                                                |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple Explanation:</b>   | A Butterworth filter is known as the 'maximally flat' filter. Other filters (like Chebyshev) have ripples (bumpy waves) in their frequency passband that distort musical harmonics. The Butterworth filter has a completely smooth, flat passband with zero ripple, making it ideal for audio. |
| <b>Project Code Mapping:</b> | <code>signal_processing.py -&gt; design_butterworth_filter(..., output='sos') &amp; sosfiltfilt</code>                                                                                                                                                                                         |
| <b>Why We Use It:</b>        | We use Second-Order Sections (SOS) to avoid numeric overflow, and zero-phase forward-backward filtering (sosfiltfilt) so audio waveforms experience zero phase delay.                                                                                                                          |
| <b>Viva Exam Q&amp;A::</b>   | <i><b>Q:</b> What is the key advantage of a Butterworth filter, and why do we use zero-phase filtering? Its key advantage is a maximally flat passband with no amplitude ripples. Zero-phase filtering (sosfiltfilt) processes forward and backward to cancel out all phase shifts.</i>        |

## Concept 14: Signal-to-Noise Ratio (SNR)

|                              |                                                                                                                                                                                                                                                                                                                                            |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple Explanation:</b>   | SNR measures how clean your sound is by comparing desired signal power against unwanted noise power. It is measured in decibels (dB). A high SNR (e.g., 25 dB) means loud clear sound with faint background noise. A low SNR (e.g., 0 dB) means the noise is just as loud as the music. If SNR is negative, the sound is drowned in noise. |
| <b>Project Code Mapping:</b> | <code>signal_processing.py -&gt; calculate_snr(clean_signal, noisy_or_processed_signal)</code>                                                                                                                                                                                                                                             |
| <b>Why We Use It:</b>        | We calculate SNR Before filtering, SNR After filtering, and Delta SNR (Improvement). This proves mathematically whether our filter actually improved audio quality.                                                                                                                                                                        |
| <b>Viva Exam Q&amp;A::</b>   | <i><b>Q:</b> How is SNR mathematically defined, and what does a positive SNR improvement indicate? <math>SNR(dB) = 10 * \log_{10}(P_{\text{signal}} / P_{\text{noise}})</math>. A positive improvement (Delta SNR &gt; 0) proves that our filter successfully reduced noise power relative to the clean signal.</i>                        |

## Concept 15: Python Architecture & Code Mapping

|                              |                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple Explanation:</b>   | The project is cleanly divided into 4 modular layers: 1) <code>audio_io.py</code> handles file reading/saving, stereo-to-mono downmixing, and speaker playback; 2) <code>signal_processing.py</code> performs all the DSP math (FFT, AWGN, Butterworth SOS, SNR); 3) <code>gui.py</code> builds the PySide6 user interface with live Matplotlib graphs; 4) <code>app.py</code> is the launcher. |
| <b>Project Code Mapping:</b> | <code>app.py</code> -> <code>gui.py</code> -> <code>signal_processing.py</code> -> <code>audio_io.py</code>                                                                                                                                                                                                                                                                                     |
| <b>Why We Use It:</b>        | Separating mathematical logic from the user interface makes the software stable, easy to read for academic evaluation, and 100% unit-testable.                                                                                                                                                                                                                                                  |
| <b>Viva Exam Q&amp;A::</b>   | <b>Q:</b> Walk me through how data flows when the user clicks 'Apply Filter'. The GUI takes <code>noisy_signal</code> , calls <code>signal_processing.apply_filter</code> with the selected filter type and cutoff, runs zero-phase Butterworth filtering, calls <code>calculate_snr</code> against <code>clean_signal</code> , and re-renders the filtered waveform and FFT plots.             |